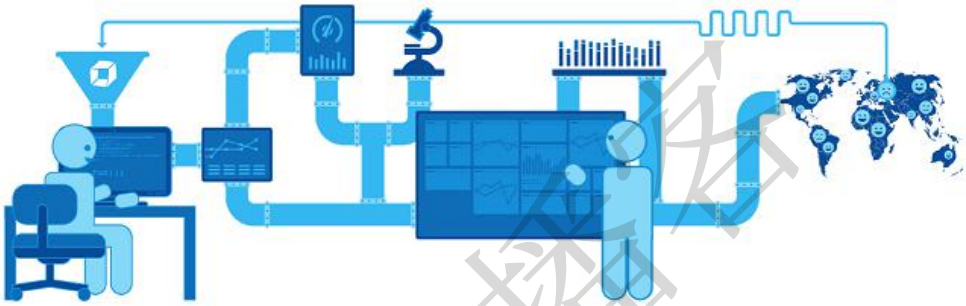
持续交付系列视频

代码配置管理

（上篇）



作者：传智播客-王老师

版本：V 1.0

文档编号：[20181205]

日期：2018年12月11日

传智播客 传智播客 传智播客

目录

绪论 1
0.1 课程基本信息 1
0.1.1 本课程目标 1
0.1.2 本课程涵盖内容 1
0.1.3 本课程适用人员 1
0.1.4 本课程组织结构 1
0.1.5 本课程书写惯例 1
0.1.6 本课程依赖技术 1
第 1 章 项目基础知识 2
1.1 生命周期 2
1.1.1 传统项目生命周期 2
1.1.2 新型项目生命周期 2
1.1.3 小结 3
1.2 项目演变 3
1.2.1 基本架构 4
1.2.2 开发框架演变 4
1.2.3 项目架构演变 4
1.2.4 小结 6
第 2 章 版本控制 7
2.1 简介 7
2.1.1 版本控制 7
2.1.2 为什么用 7
2.1.3 小结 8
2.2 版本控制种类 8
2.2.1 分类及使用 8
2.2.2 小结 8
第 3 章 分支管理 9
3.1 基础知识 9
3.1.1 常见术语 9
3.1.2 分支概念 9
3.1.3 代码开发流程 10
3.1.4 小结 10
3.2 分支开发 10
3.2.1 分支开发-主干发布 11
3.2.2 分支开发-分支发布 12
3.2.3 小结 12
3.3 主干开发 12
3.3.1 主干开发-分支发布 13
3.3.2 主干开发-主干发布 13
3.3.3 生产案例 14
3.3.4 小结 14
第 4 章 开发模型 15
4.1 软件开发模型 15
4.1.1 边做边改模型 15
4.1.2 瀑布模式 15
4.1.3 迭代模型 16
4.1.4 增量模型 17
4.1.5 快速原型模型 18
4.1.6 螺旋模型 18
4.1.7 敏捷开发模型 19
4.1.8 精益模型 20

传智播客

绪论

0.1 课程基本信息

0.1.1 本课程目标

本课程的教学目标，是通过对软件项目交付过程的理解，来学习配置管理中的核心部分--代码配置管理内容。本课程主要有上下两篇构成：

上篇主要讲述代码配置管理的知识，偏重于解决方案和理论

下篇主要是代码配置管理中的工具实践，包括git、gitlab、github及综合实践

0.1.2 本课程涵盖内容

本课程的基调是中立的：即通过代码配置管理的学习，了解并应用项目开发的常见方式，它不对任何提供商的产品、服务和技术做过多的推广。

如果您是首次接触代码配置管理领域，建议您将本课程作为出发点，在理解相当的基础知识之上，然后再去快速了解其他平台的资料或相关视频。

0.1.3 本课程适用人员

适合本课程的学习人员有：

对配置管理有兴趣的IT从业者或相关人士

对配置管理有兴趣的开发、运维、测试等相关人员

需要对配置管理境深入理解的人员

0.1.4 本课程组织结构

本课程的组织结构如下：

第 1 章 项目基础知识

本章由 2 部分组成：生命周期和项目演变。

生命周期：

通过对传统和新型的项目生命周期介绍，理解代码配置管理在项目研发过程中的定位。

项目演变：

通过对项目架构的部署演变，了解项目运营过程中，代码配置管理的变化。

第 2 章 版本控制

本章通过版本控制的种类和特点，了解常见的版本控制工具。

第 3 章 分支管理

本章在第 2 章的基础上，学习多版本产品的常见发布流程方法。

第 4 章 开发模型

本章在第 3 章的基础上，结合第 1 章项目演变的过程，对项目开发的常见模型进行介绍。

0.1.5 本课程书写惯例

图和表格

本课程使用了大量的图和表格，方便大家在学习的时候快速的理解和记忆

关键点

为了便于快速的查阅和记忆，本课程在每一节的末尾都添加的了内容小结，强调了相应小节部分的主要内容。

0.1.6 本课程依赖技术

项目基本开发技能

传智播客

第 1 章 项目基础知识

1.1 生命周期

学习目标：

了解 项目生命周期

1.1.1 传统项目生命周期

世间万物皆有其生命，软件项目也是如此。随着互联网的发展，软件项目的生命周期也发生了很大的变化，为了更好的让大家理解软件项目，项目生命周期有狭义[具体]、广义[缘起/缘灭]之分，我们一般所说的项目生命周期主要指的是狭义的项目生命周期，我们以传统的软件项目为例进行介绍。

传统项目生命周期

对于传统软件项目来说，它主要包含以下五个阶段：

1 调研阶段

目的：居安思危

人员：相关人员，侧重于产品经理

节点：多角度思路/方案，最后领导拍板

2 设计阶段

目的：方案可视化

人员：产品团队主导，开发、测试、运维参与

节点：产品需求文档、项目里程表

3 开发阶段

目的：方案运行

人员：开发团队为主，运维团队参与

节点：阶段项目正常运行

4 测试阶段

目的：保证项目功能完善

人员：测试团队为主，运维、开发参与

节点：项目功能符合要求

5 运营阶段

目的：项目部署+运行维护

人员：运营团队为主、开发/产品团队参与

节点：项目终止、功能迭代等

1.1.2 新型项目生命周期

对于传统的软件项目来说，由于项目组织和团队组织的特性导致传统的软件项目开发的过程中，文件的主导作用很大。而新兴的软件项目来说，项目的软件架构和团队组织架构都发生了千变万化，而且新旧项目生命周期的关注对象也发生了很大的变化，在新型软件项目管理中，人/团队沟通协作占到了很大的主导作用。

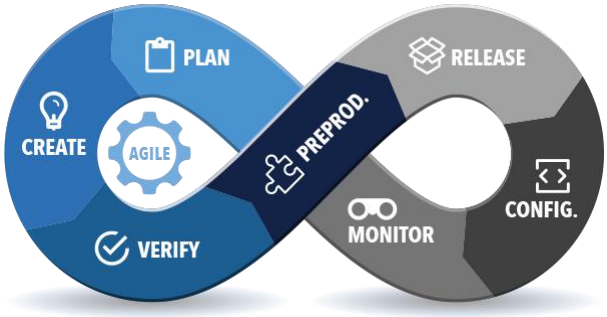
项目阶段

软件架构：

对于新型软件项目来说，项目在开发的过程，跟从项目整体功能入手的传统软件不一样。新型软件项目在开发前一般会对项目进行软件功能架构的拆分，拆分成多个彼此独立的子功能项目，子项目彼此间约束很少，每个子功能项目都可能基于传统的项目的阶段特点对项目团队进行合理拆分。

团队架构：

由于软件架构的特性，每个子功能项目可以独立组件研发团队，新型软件项目研发团队主要是靠人/团队沟通协作来主导项目的开发进度，这就对人员的综合能力要求很高，相应的对负责人的专业技术能力要求更高了。

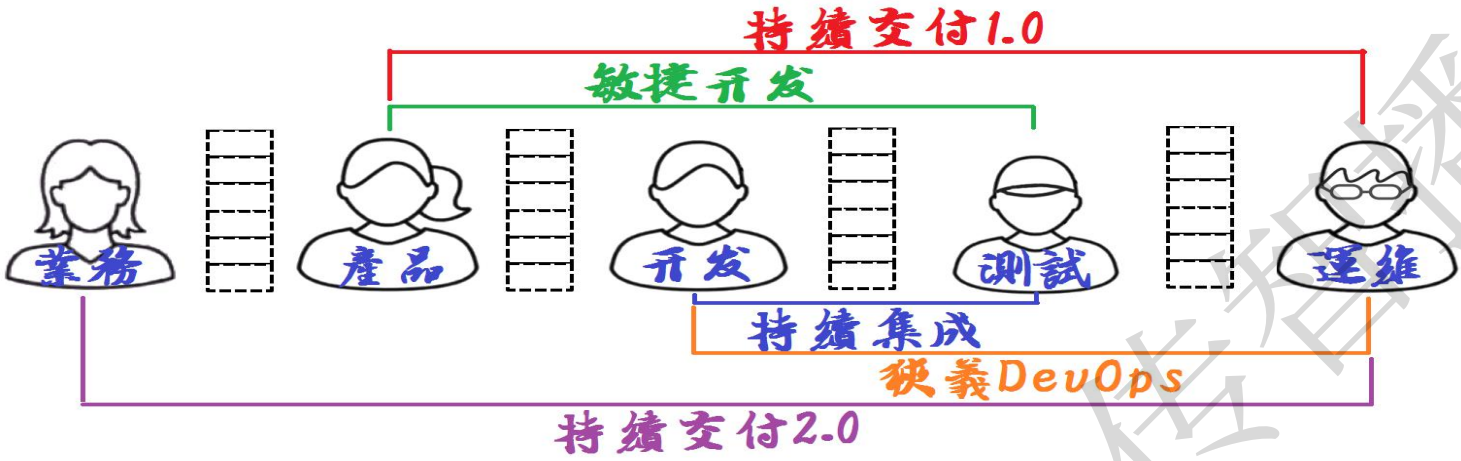


项目实施：

基于上面两方面内容的变动，导致我们项目编码的各个阶段发生了非常大的变化。

常见术语：

由于软件架构导致的组织变化，软件开发的过程中出现了非常多的新鲜名称：



常见词语解决的问题：

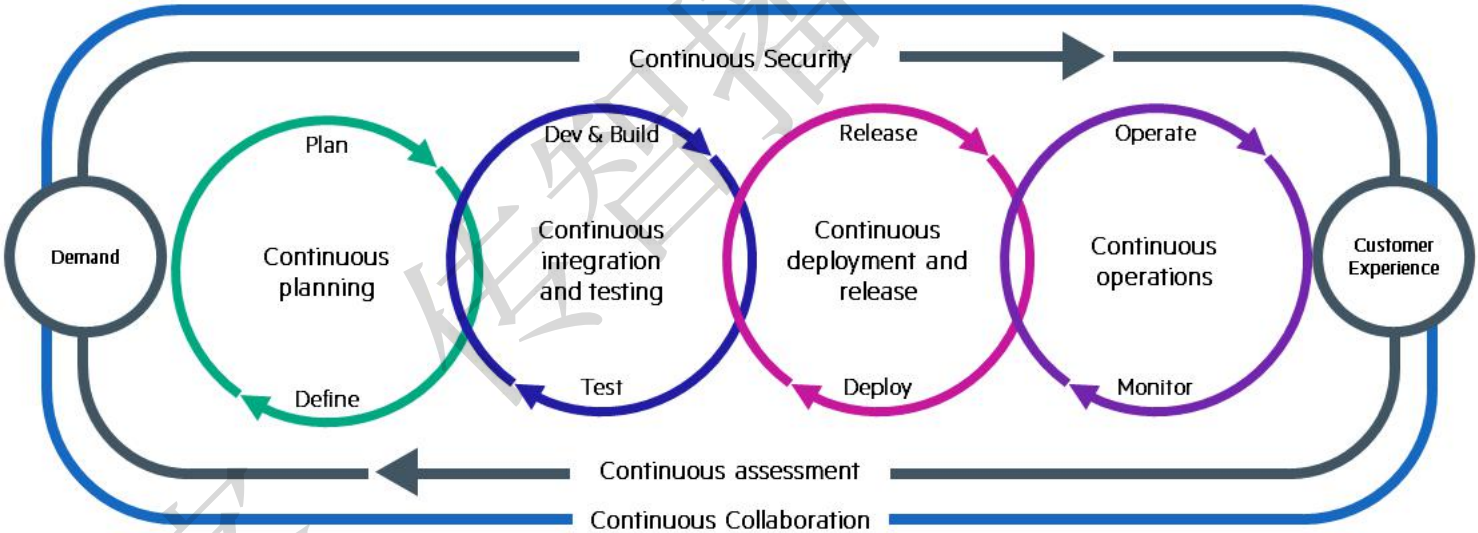
持续集成：开发团队和测试团队之间的无障碍协作沟通

敏捷开发：在持续集成的基础上，产品团队和研发团队无障碍协作沟通

狭义DevOps：在持续集成的基础上，研发团队和运维团队无障碍协作沟通

持续交付1.0：各团队的无障碍协作沟通，实现项目产品的端到端交付能力,体现的是产品交付的完整过程

新型软件生命周期效果图



1.1.3 小结

本节内容小结：

传统项目生命周期：调研-设计-开发-测试-运营

新型项目生命周期：从软件架构、团队架构来理解

常见术语：持续集成、敏捷开发、狭义DevOps、持续交付

1.2 项目演变

学习目标：

了解 项目的基本结构

了解 项目开发框架演变

了解 项目架构演变

1.2.1 基本架构

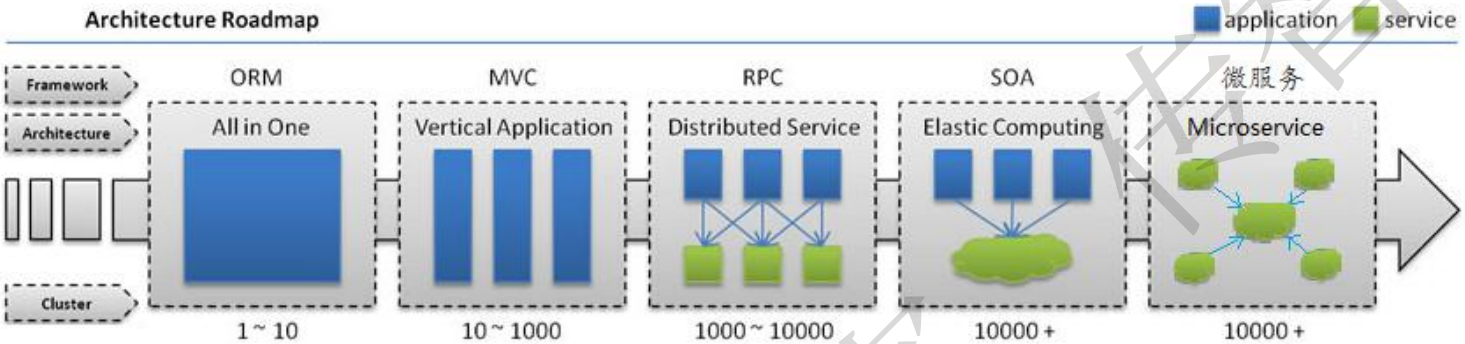
项目的架构

一般来说，任何一个项目至少最核心部分来组成：

访问层、数据层、存储层

1.2.2 开发框架演变

项目开发框架演变



1.2.3 项目架构演变

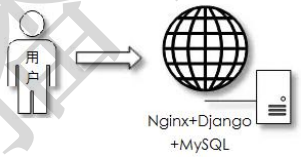
初级阶段

单体阶段

常见场景：项目初期

部署特点：所有应用服务都在一台主机

应用特点：开发简单

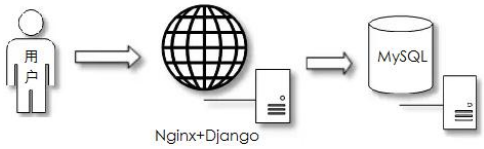


应用/数据分离阶段

常见场景：项目初期，用户访问数据库有压力

部署特点：应用和数据库单独部署

应用特点：开发简单

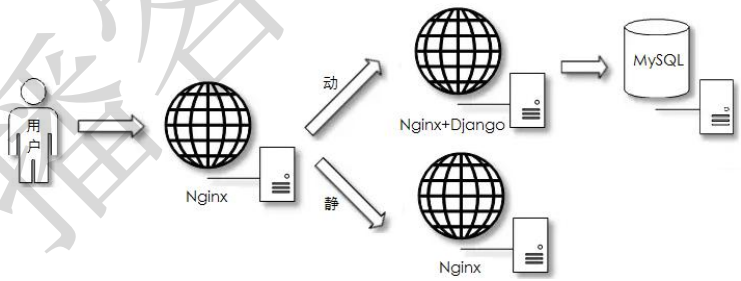


页面动静分离阶段

常见场景：项目初期，用户访问页面有压力

部署特点：剥离用户读请求和写请求操作

应用特点：开发简单



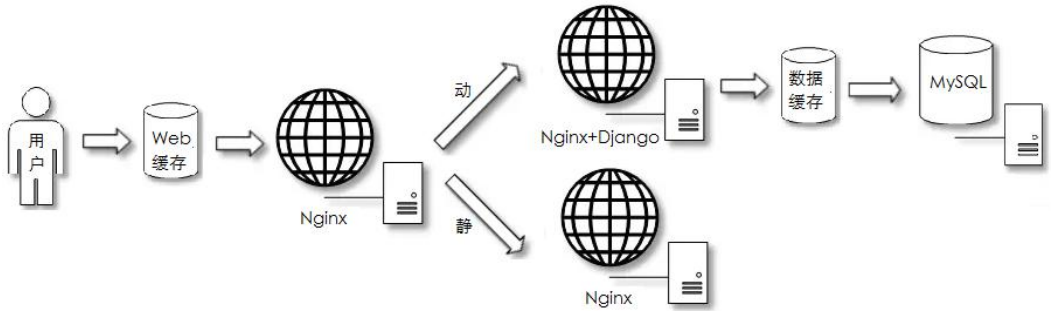
页面/数据缓存阶段

常见场景：项目初期，用户访问有压力

部署特点：代理和数据库前面增加缓存组件

应用特点：开发简单

传智播客



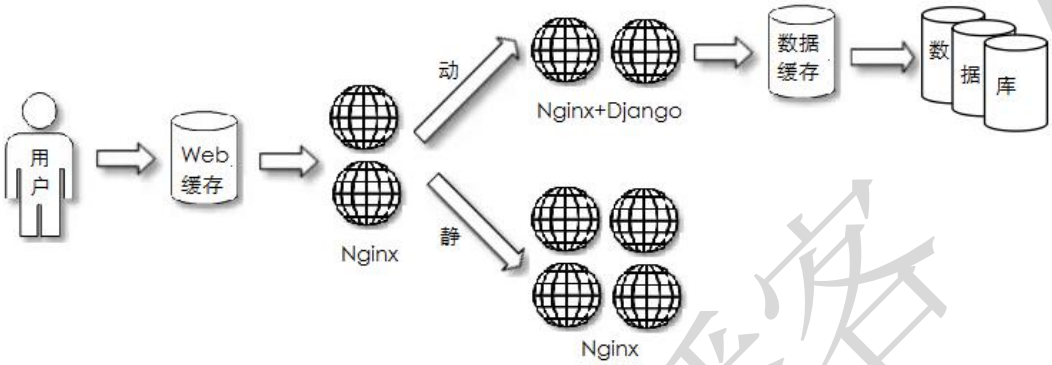
中期阶段

应用服务集群阶段

常见场景：项目初期，用户访问有压力

部署特点：应用服务所在主机做集群负载均衡

应用特点：业务中等

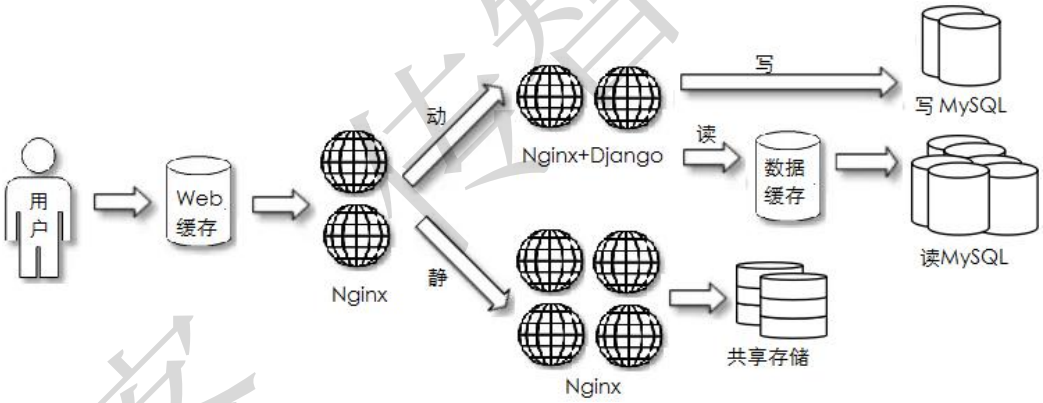


数据库读写分离化

常见场景：项目初期，用户访问数据有压力

部署特点：对数据库集群做读写分离，静态文件做共享存储

应用特点：业务中等



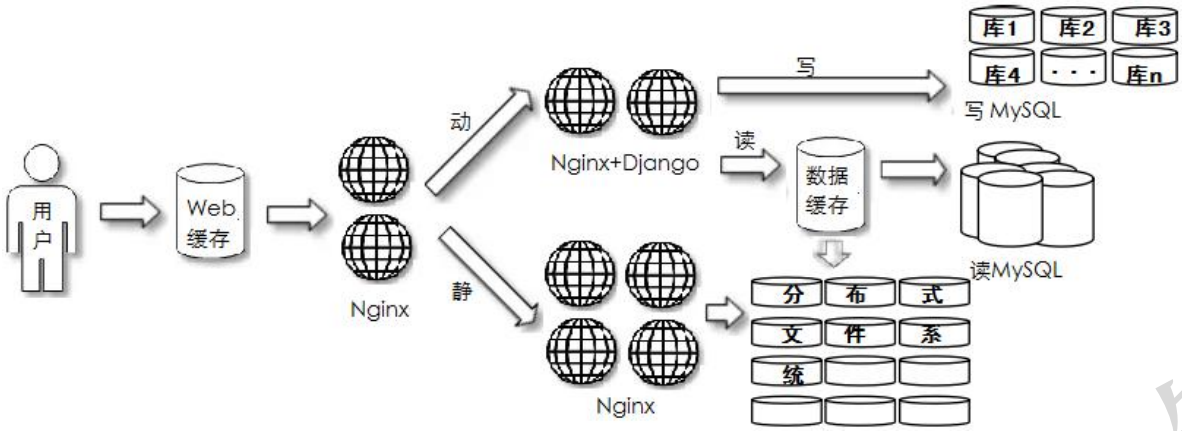
存储分布式

常见场景：项目中期，数据存储有压力

部署特点：对数据库分库/分表扩展，数据文件使用分布式存储

应用特点：业务中等

传智播客

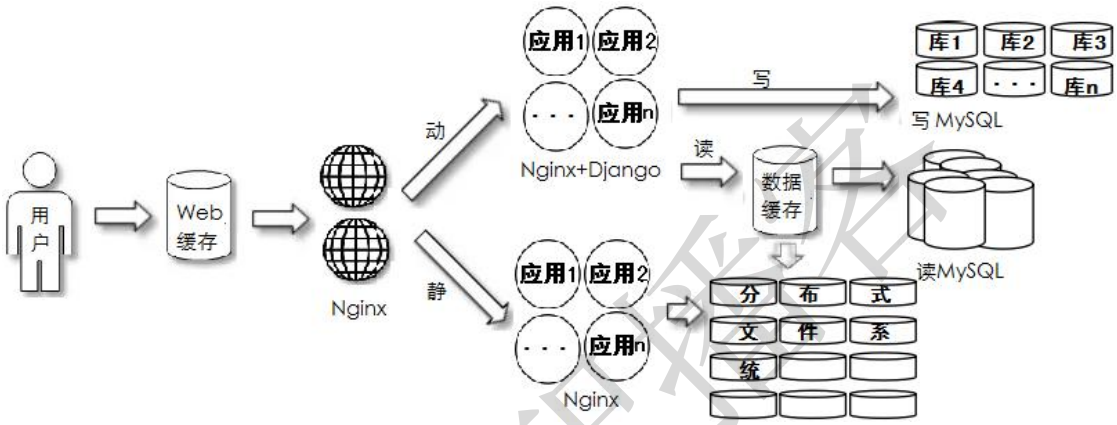


业务应用拆分

常见场景：项目中期，业务访问/团队管理有压力

部署特点：项目应用进行拆分

应用特点：业务复杂



中后期阶段

业务拆分

常见场景：项目中后期，业务处理有压力

部署特点：所有功能以服务形式单独部署，引入配置管理管理中心、消息中间件，搜索引擎等功能

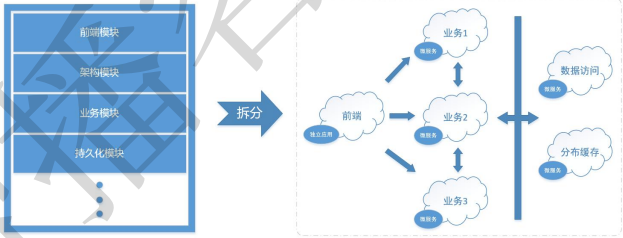
应用特点：业务复杂

微服务阶段

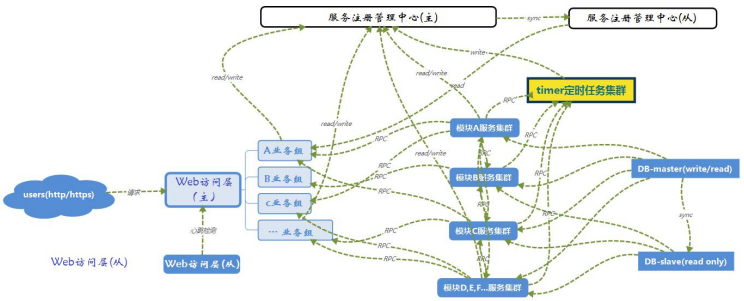
常见场景：项目后期，精益求精

部署特点：所有服务都可以自由部署

应用特点：业务复杂



单体架构　　微服务架构



1.2.4 小结

软件项目基本架构：web访问层、数据层、存储层

开发框架演变：单体、MVC、分布式、SOA、微服务

项目架构演变：

初期

单体阶段、数据分离、页面分离、增加缓存

中期

服务集群、读写分离、存储分布、应用拆分

中后期

业务拆分、微服务

第 2 章 版本控制

2.1 简介

学习目标：

了解 版本控制作用及特点

了解 版本控制主要功能

2.1.1 版本控制

版本控制

版本控制(Revision/Version control),团队项目产品开发的过程中，项目文件代码保持一致性的一种方式。

示例：最简单的版本控制就是保存项目内容的一个备份，编号为"A"，然后基于原始项目进行修改，修改后保存为版本"B",保留软件不同状态的数份copy，并且适当编号。许多大型开发案都是使用这种简单技巧。虽然这种方法能用，但是很没效率。一是因为保存的数份copy几乎完全一样，也因为这种方法要高度依靠开发者的自我纪律，从而导致错误。

版本控制系统

版本控制系统(Revision/Version control system)，就是能自动实现特定/完全版本控制功能的一种软件实现平台，它是文件多版本状态共存的一种机制。

示例：软件初版等级"A"，基于该版本做了一些修改后，提交后版本等级改为"B"，以此类推逐渐出现更多的版本，版本控制的一个最重要的功能就是历史记录追踪，它基于版本号，让项目整体内容恢复到任一版本状态，这种功能在防止产品进行未知功能设计失误时特别重要。

2.1.2 为什么用

版本存储

常见的版本存储方式就是在本地多次备份不同版本的文件，即使按照通用的命名格式保存后，还需要花费大量的时间来分析整理这些备份文件，而且这种操作很容易出错，而且经常性的不知道为什么保存，保存了什么变动的内容，因为我们很少花更多的时间去记录和观察每一个重要的变化

版本控制系统就完美的解决了我们的问题，每当你提交一次对项目新的改动时，他会基于最原始的文件，保存每一个细节的变化到一个版本中，可以帮助我们很好地了解相邻版本间的变动关系。

而且版本控制系统有撤销的功能，我们可以基于某个版本号，撤销所有/部分的变动信息，回到当时的文件状态。在项目的每一个重要阶段，认识和正确地使用撤销功能会让我们的工作变得非常轻松。

协同合作

基于传统方式修改项目代码的时候，必须告知团队中的其他人，我在干什么，防止他们和我冲突，而这不可能的，所以当我好不容易编辑完文件后，发现该文件被人删了，感觉很不舒服。

有了版本控制系统，团队每一个成员都可以自由的修改代码文件，进行团队的协同工作，版本控制系统可以帮

我们将所有改动内容合并保存为一个版本，即使某些功能代码意外丢失，也可以通过版本系统找回。

2.1.3 小结

版本控制：生产中管理多版本项目文件的一种方式

主要功能：存储、协同工作

2.2 版本控制种类

学习目标：

了解 版本控制种类及其工具基本情况

随着互联网的发展，软件产品的更新迭代越来越快，软件产品代码的版本控制系统也发生了千变万化，既有开源的，也有商用的，而且都是针对各种不同的应用环境设计的。目前市场上出现比例较高的版本控制系统主要有两类：集中式版本控制系统和分布式版本控制系统。

2.2.1 分类及使用

集中式版本控制系统

集中式版本控制系统：产品项目的标准代码集中放在中央代码服务器的，我们工作时候，必须连接中央代码服务器并获取最新版本代码，进行编写或者更改代码并检查完毕后，推送到中央代码服务器。该版本控制系统受网络限制比较大。

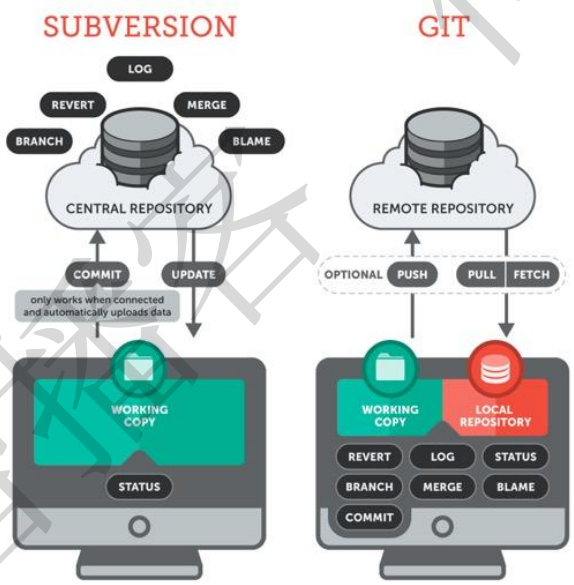
常见的工具：CVS(Concurrent Versions Control System)，**SVN**(Subversion)，VSS(Visual Source Safe)、...

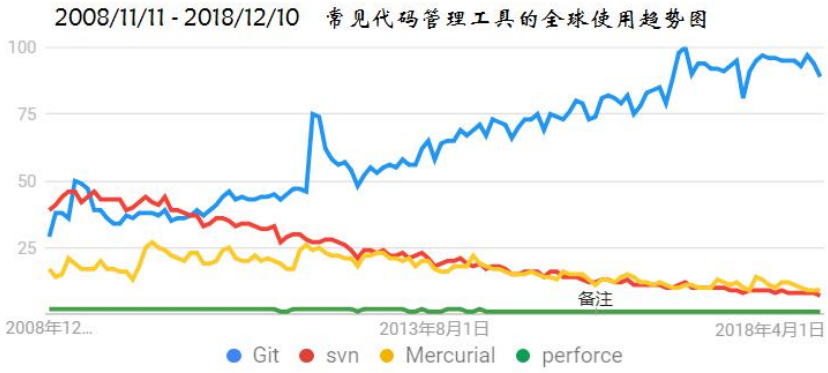
分布式版本控制系统

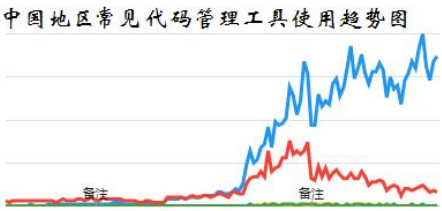
分布式版本控制系统：分布式版本控制系统没有中央代码服务器的。团队中每个人的电脑就是一个完整的代码版本库，我们都在自己的电脑上进行工作，即使没有网，工作也不受影响，团队成员的所有代码最终会合并在一起。因为每个人都可以在本地对同一个文件进行修改，所以在代码合并的时候，会受到一定的影响。

常见的工具：**Git**、 Mercurial、Monotone、Bitkeeper、...

常见工具使用情况







2.2.2 小结

版本控制分类：集中式(svn)和分离式(git)

两类版本控制区别：管理时候是否实时处于网络连接状态

第 3 章 分支管理

3.1 基础知识

学习目标：

了解 代码配置管理中的常见术语

了解 代码配置管理的实现方式--分支

记住 标准分支使用流程

3.1.1 常见术语

基本术语

仓库： 代码文件存储的位置，常见的有工作目录->缓存区->本地仓库->远程仓库。

版本： 项目开发过程中，使用版本来记录文件在每个阶段的变化内容。

分支： 保存文件的一种方式，而且分支之间实现了资源隔离。

动作术语

获取： 从上级仓库位置获取文件到当前位置。

提交： 将修改好的文件提交到上级仓库位置。

合并： 将某个仓库位置的文件和本地位置的文件组合在一起。

冲突： 合并的过程中，由于文件、语意方面的不一致，导致不能正常合并。

解决： 对于冲突的场景，进行人为干预，最终达到正常合并的条件。

其他术语

hook： 触发器，我们在做一件事情的时候，会自动触发已定义好的事件。

锁定： 对仓库中文件进行安全保护的一种权限设置。

3.1.2 分支概念

分支简介

在一个代码基础上创建分支的能力是版本控制系统最重要的特性。这个操作实在版本控制系统中对选定的代码内容创建一个副本，所以分支其实就实现了一种项目代码资源隔离的方式，保证我们在项目开发的过程中，组员间之间工作互相不受干扰。分支的主要目的就是帮助项目并行开发。

由于项目开发过程中的代码资源隔离，那么我们就可以基于这种资源隔离的特点，实现项目的不同状态管理，这就出现了多种角色分支。

我们按照日常的使用频率和角色地位的不同，将其分为常用性分支和临时性分支。

常用性分支

主分支(master/trunk)

每个代码库有且仅有一个主分支。它是版本库初始化以后自动建立的，默认就是在主分支在进行开发。

开发分支(develop)

项目代码的日常开发所在的分支

临时性分支

功能分支(feature)

为了开发某种特定功能，从Develop分支上分出来的。临时开发完成后，要再并入Develop

预发布分支(release)

预发布分支，它是指发布正式版本之前(合并到Master分支之前)，进行版本代码测试的临时性分支，测试通过后，就正式发布代码。代码发布完毕后，需要合并到develop和master分支上

修补bug分支(fixbug)

软件项目正式发布以后，难免会出现bug，就临时创建一个bug分支来进行功能修补，功能修补完毕后，在将最终代码合并到Master和相应的Develop。

传智播客

使用分支的原因

团队中使用分支的主要原因有以下几种：

物理上：基于项目代码目录结构配置，即为了文件、组件、和子系统而分支。

功能上：基于项目多功能配置，即为特性，逻辑修改、缺陷修复、功能递增。

环境上：基于项目多运行环境，即为开发、测试、预发布、线上环境等。

组织上：基于项目团队的工作量，即为活动/任务、子项目、角色、群组等。

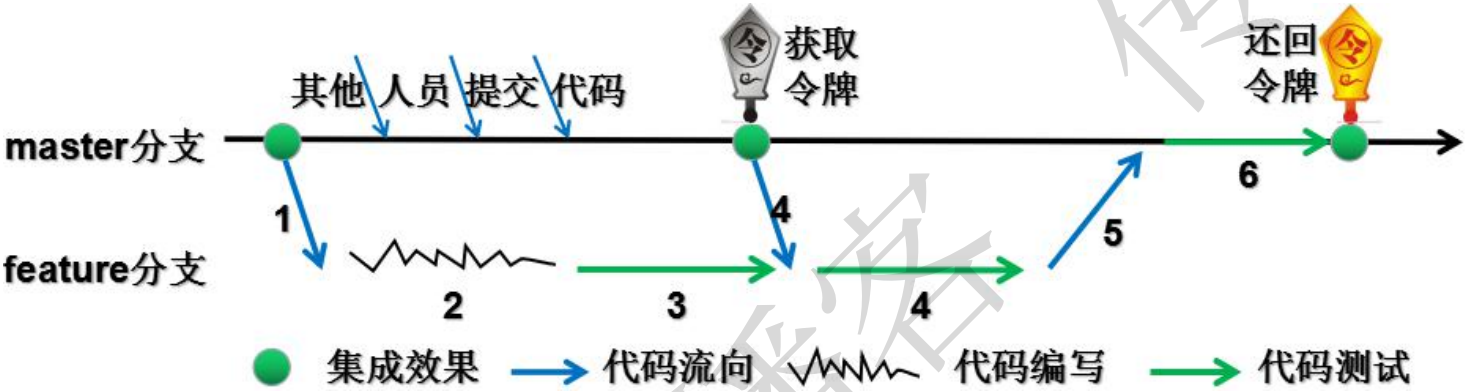
流程上：基于项目团队的工作行为，即为支持不同的规范、流程、状态等。

这些分类的分支并不互相排斥，只是在不同场景下我们未来创建分支的一种理由，方便我们的团队工作。

3.1.3 代码开发流程

代码开发标准流程

项目团队在启动项目开发工作之前，应该共同约定的一个行为规范，避免团队协作开发的过程中出现意外，只要是与代码相关的操作，我们的代码开发工作步骤就应该是这样：



这张图是持续集成六步提交法。这是团队开始启动这个试点项目时共同约定的一个行为规范。只要是与代码相关的操作，你的工作步骤就应该是这样的。

1 从主版本库master中切到本地一个功能分支feature

注意：必须在主分支持续集成状态为绿色时(代码无问题)，拉取分支

2 基于新功能或bug修改本地功能分支代码

注意：本地编写代码，按照自己的速度，保证质量

3 对2完成的代码进行本地功能测试

注意：侧重于组件接口测试

4 验证无问题后，从主版本库中拉取没问题的最新代码，合并到本地功能分支上，并再次执行本地验证

注意：

因为本地开发过程中，master分支可能被别人提交过，所以我们提交代码时候不能和新master冲突

本地测试时间一般不长，如果本地测试过程中，又有人提交新代码到master，必须重复4动作

5 将二次验证后的本地feature分支代码提交到代码主版本库master

注意：一定要先本地测试成功，而且master分支跟刚才拉取的状态一致

6 代码提交者基于自己提交后的主分支代码触发的持续集成线上构建，直至其成功。

注意：如果构建失败，必须做最高优先级任务对其进行修复

3.1.4 小结

分支种类：常见(master/develop),临时(feature、release、bugfix)

为什么用分支：物理、功能、环境、组织、流程

代码提交流程：拉取、开发、测试、拉取+测试、合并、测试、发布

3.2 分支开发

学习目标：

了解 两种分支开发的流程及其特点

传智播客

简介

分支开发(branch development)。它的特点就是项目的功能开发工作主要在从master主分支切出来的feature分支上。根据功能代码开发完毕后的项目发布，就出现了两种不同的发布模式：

分支开发-主干发布

分支开发-分支发布-合并回主干

案例需求：

n 月 1 日 A项目发布1.0版本

n 月 2 日 A项目着手开发1.1 版本的新功能

n 月 7 日 线上运行A项目的1.0版本出现问题，并与当日修复并发布1.01版本

n 月 16日 线上运行A项目的1.0版本出现问题，并与当日修复并发布1.02版本

n 月 20日 A项目完成新版本功能，并发布1.1版本

注意：

1.01和1.02的修复代码过程不影响1.1正常发布

1.1 版本包含 1.01和1.02的修复代码功能

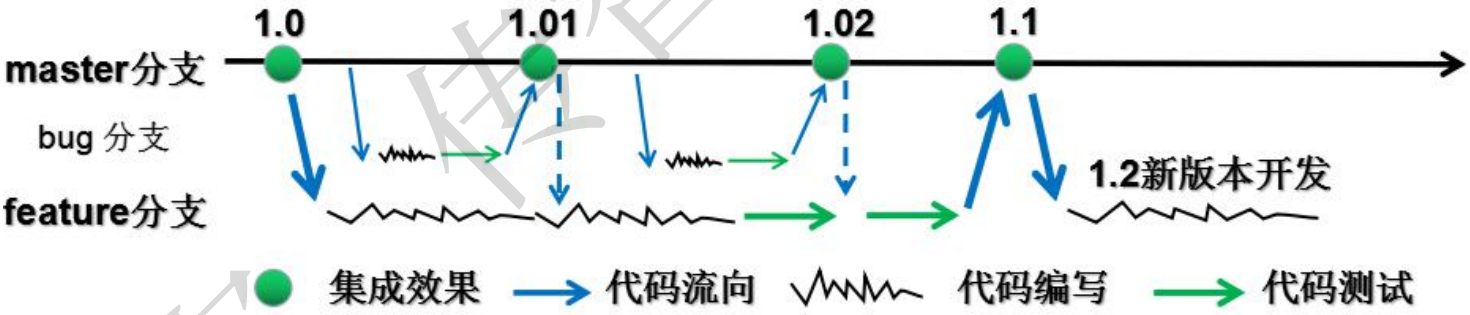
3.2.1 分支开发-主干发布

分支开发-主干发布简介

分支开发-主干发布。它的特点就是项目的功能开发工作在从master主分支切出来的feature分支上，代码开发完毕后，经过功能测试没有问题后，在正式代码发布之前，先从master主分支获取最新/最完整的代码，合并到当前的feature分支上，进行整合功能测试，测试通过后，就将代码合并到master主功能分支上，发布代码的时候，只需要打一个release标签即可。如果进行下一个功能版本开发，那么就从master分支切出一个新的feature分支即可。

如果线上代码出现bug，那么就从master分支上切出来一个bug分支，修复该bug即可。

代码开发效果



代码开发原则

1 只能在功能feature分支上开发，不能直接在master主分支上开发

2 feature分支代码开发完毕后，代码功能必须通过测试

3 feature分支在合并到master分支前，获取最新的master分支内容，合并后进行功能测试，再合并到master

4 feature分支合并到master主分支后，应该停用该feature分支，最好删除。

5 master主干上的每一次发布都是做了一个release标签而不是分支。

优势：

常见的一种代码开发方式。

团队人员在自己的feature分支工作，隔离其他工作的变动

多个功能使用feature分支并行开发，互不影响

因为都是每次功能代码发布都是在之前基础上，所以代码内容调整起来比较容易，即使有冲突也可以快速解决

劣势：

feature分支最终要合并到master分支，若feature分支间代码有交互，导致master合并冲突（文件、语意）

feature分支代码测试，需要单独建立测试流水线，浪费资源，而且并未真正实施项目集成测试。

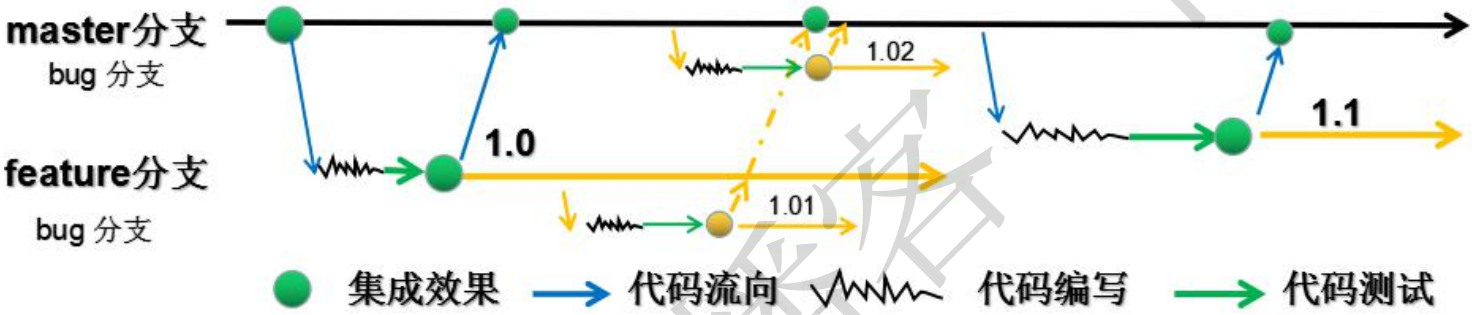
3.2.2 分支开发-分支发布

分支开发-分支发布简介

分支开发-分支发布的特点就是项目的功能开发工作在从master主分支切出来的feature分支上，代码开发完毕后，经过功能测试没有问题后，然后直接在当前feature分支上进行代码发布，发布后，将feature分支的代码合并到master主分支上。如果进行下一个功能版本开发，那么就从master分支再切出一个新的feature分支即可。

如果线上代码出现bug，那么就可以直接在feature分支上修复，修复后，将代码合并回master分支；如果bug功能很重要，可以单独从master分支上再切一个bug分支来修改。

分支开发-分支发布流程



代码开发原则

1 只能在功能feature分支上开发，不能直接在master主分支上开发。

2 feature分支代码开发完毕后，代码功能必须通过测试。

3 feature分支代码功能通过测试后，直接进行代码发布。

4 feature分支代码发布后，将最终代码合并到master主分支，该feature分支作为版本可以一直存在下去

优势：

最常见的代码开发方式

多个功能使用feature分支并行开发，互不影响

可以选择指定的feature分支进行代码发布，不会被其他功能阻塞。

劣势：

feature分支最终要合并到master上，若feature分支间代码有交互，导致合并冲突（文件、语意）。

feature分支发布后容易忘记合并回master主干。

feature分支代码测试，需要单独建立测试流水线，浪费资源，而且并未真正实施项目集成测试。

常见场景：

开源项目、多产品线的项目

3.2.3 小结

分支开发分类：分支开发-主干发布、分支开发-分支发布

主干发布流程：拉取-开发-测试-拉取-测试-合并-发布（标签）-循环

分支发布流程：拉取-开发-测试-发布-合并-循环

3.3 主干开发

学习目标：

了解 主干开发的两种流程及其特点

简介

主干开发(trunk-based development TBD)。它的特点就是项目的功能开发工作主要在从master主分支上。根据功能代码开发完毕后的项目发布，就出现了两种不同的发布模式：

主干开发-主干发布

主干开发-分支发布

3.3.1 主干开发-分支发布

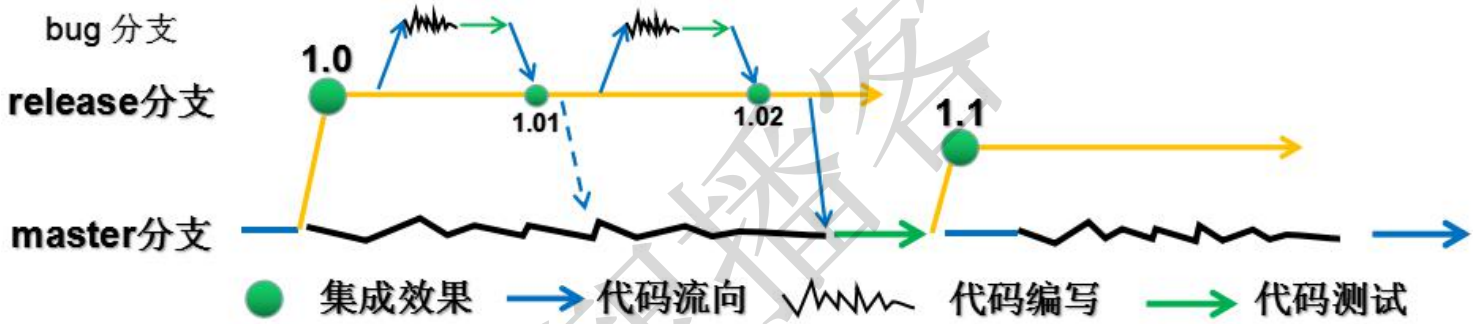
主干开发-分支发布简介

主干开发-分支发布的特点就是项目的功能开发工作在master分支上，代码开发完毕后，经过功能测试没有问题后，从master主分支上切出一个release分支，作为项目代码版本发布的专用分支，而master分支还继续做项目下一个功能版本的开发工作。

如果线上代码出现bug，那么就在release分支上修复就可以了，修复后的代码最终要合并到主干上，只有非常严重的缺陷修复才会从master合并到release分支上。

因为主干代码一致处于可发布状态，所以更容易发布，而且，我们可以多个发布版本同时存在。

主干开发-分支发布效果



主干开发-分支发布原则

1 项目所有主要/核心功能代码的开发工作都在master分支上。

2 项目重要功能发布的时候，从master单独切出release分支，作为该功能的正式发布分支。

3 项目正常运行过程中出现功能(bug)问题，在相应的release分支进行bug的修复工作。

4 项目新功能在测试发布前，必须先合并上一个release分支代码。

5 缺陷(bug)修复，最终需要合并到主干，但有些特定修改，是不需要合并到主干的，合并准确的文件到主干。

6 对于不能合并到主干的情况，视情况再拉一个分支专门保存该内容。

优势：

主干开发的过程中，关于分支合并的工作量很少，所以整体比较简单

劣势：

新功能代码在master主干上开发，若主干代码达不到稳定的标准，不可以进行项目发布

使用场景：

比较适合传统软件产品的开发模式的项目、多产品线的项目

被广泛的应用于开源项目

强烈推荐、业界标准

3.3.2 主干开发-主干发布

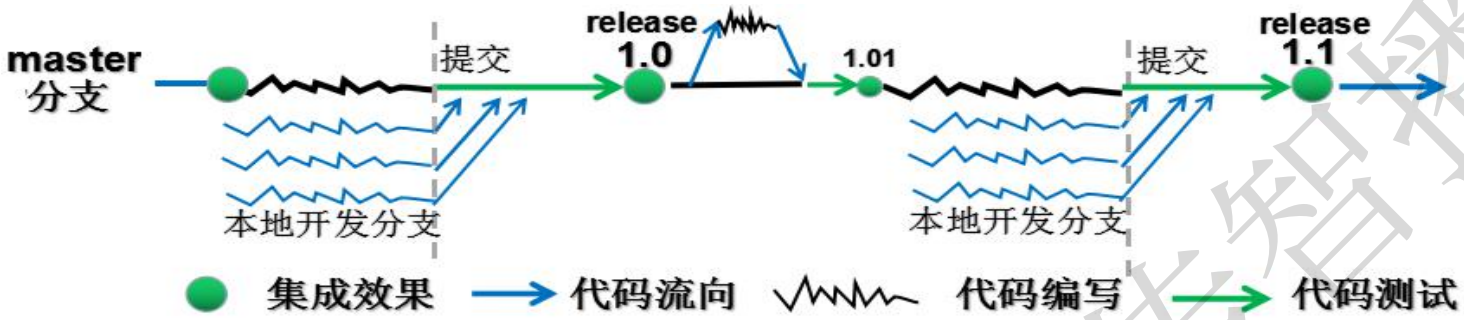
主干开发-主干发布简介

主干开发-主干发布的特点就是项目的功能开发工作在master分支上，代码开发完毕后，经过功能测试没有问

题后，在master主分支上打release标签作为项目代码版本进行发布。发布完毕后，master主分支还继续做项目下一个功能版本的开发工作。

如果线上代码出现bug，那么就在master分支上修复就可以了。

主干开发-主干发布效果



主干开发-主干发布原则：

1 项目所有主要/核心功能代码的开发工作都在master分支上，开发完毕后，在master分支上进行集成测试。

2 项目代码测试通过后，通过打标签的方式，以标签的方式进行代码发布。

3 项目正常运行过程中出现bug，在master分支进行bug的修复工作，修复代码通过测试后，打标签发布。

优势：

效率最高的一种项目开发方式。

项目代码发布前，需要进行主干集成测试，代码冲突易于提前发现

master主干代码安全稳定，每次测试通过后，都可以随时发布。

集成测试常见的配套措施：每日集成(编译部署测试)，代码检查，单元/接口/界面自动化测试。

劣势：

新功能代码在master主干上开发，若主干代码达不到稳定的标准，不可以进行项目发布

master上主干开发有先后，有未完成的功能但又需要发布时，需要能隐藏未完成部分。

为了避免以上情况，有三种缓解方法：

1 功能拆分，小批量频繁发布；

2 后端先行，ui或功能入口发布；

3 功能开发，配置决定功能

3.3.3 生产案例

根据DevOps研究评估组织(DORA)连续多年对互联网公司的IT效能的调研，全球数千家公司参与了该项调查，其中关于代码配置管理的内容，值得我们思考。

该组织调查了基于主干开发在持续交付过程中所起到的作用，根据以往数年的经验，在高效能研发团队中，相比长期存在的特性分支，基于主线的小批量研发分支更加受到欢迎，行业中的很多先驱者倾向于把工作至于分支上。调查结果表明，以下开发实践可以显著帮助软件交付变得更加高效：

1 每天向主干合并一次代码

2 让分支生命周期尽量短(少于一天)

3 同一时间少于三条的活跃分支

3.3.4 小结

分支开发分类：主干开发-分支发布、主干开发-主干发布

分支发布流程：开发-测试-拉取-发布-循环

主干发布流程：开发(拉取)-测试-发布-循环

第 4 章 开发模型

4.1 软件开发模型

学习目标：

了解 多种软件开发模式及其特点

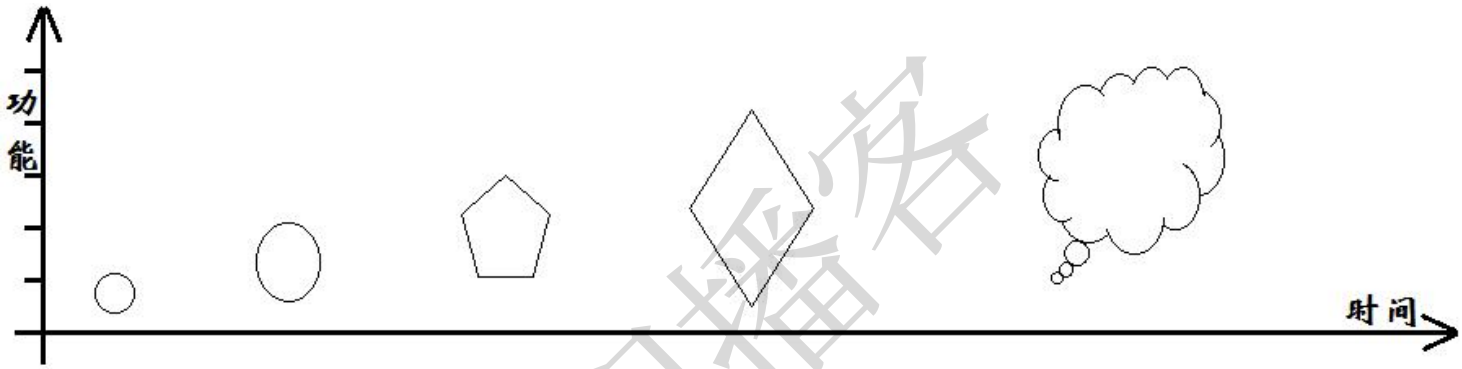
在项目正常运营过程中，关于项目的功能迭代是一个非常重要的内容，主要考虑的是开发团队的工作模式

4.1.1 边做边改模型

边做边改模型 (Build-and-Fix Model)

边做边改模型很常见，很多公司由于项目、资金、人员、投资等方面的原因，无限制的压缩产品的交付周期，根据制定的临时需求计划[几乎没有规划]就进行项目开发，通过快速的创建调试后，交付给用户产品，如果运行起来后，不满足用户需求，那么就在当前产品的基础上修改，直到达到指定用户的满意为止。

简单来说就是：走一步，看一步，再走一步，...



优势：

这种作坊式的软件生产方法，好处就是快速的产出效益，企业回笼资金快。

劣势：

1 野草式发展：无计划的发展，随着软件结构的无规律修改，产品会越来越糟，最终导致无法继续修改。

2 质量无保证：开发过程无考虑测试和可维护性，也没有任何文档，软件的维护十分困难。

场景：

项目初期使用最频繁

逻辑不需要太严谨的小项目

4.1.2 瀑布模式

瀑布模式(Waterfall Model)

瀑布模型提供了软件开发的基本框架。它在传统的软件生命周期的基础上，以文档计划来推动项目生产的一种软件开发模式。其过程是：每个阶段(B)只接受上一阶段(A)的工作成果(A-Pro)，同时评审该内容是否适合当前阶段(B)工作的基本要求。

若不符合，则返回前一阶段(A)，甚至更前面的阶段进行确认；

若符合，则继续完成当前阶段(B)的工作内容,并将生成工作成果(B-Pro)。然后继续刚才"接受-验证-继续"的循环，直到产品完成交给用户。

优势：

因为瀑布模型的每个阶段的边界划分很清楚，而且以线性的顺序依次完成。如果我们将阶段间的联系划分的更细一些，意味着我们在阶段性交接工作成果的时候没有更多的验证约束。即使交接的工作成果有问题，我们只需要纠正一下即可。

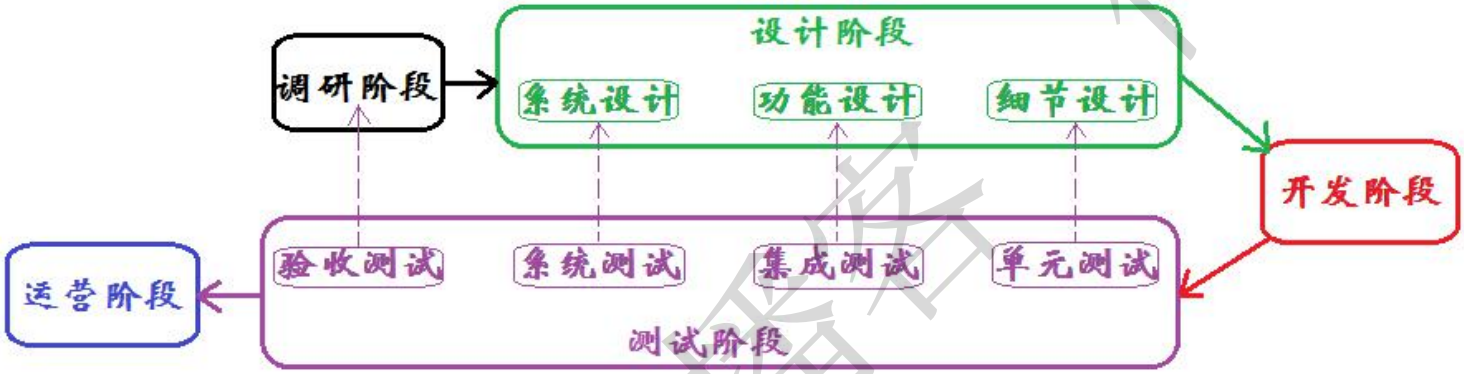
瀑布模型有明确的需求分析，层次性的开发思路，我们只需要按部就班的完成各阶段的工作内容就可以，它是一种理想化的开发模型，非常适合过程管理。

劣势：直接影响正常的产品交付日期

瀑布模型的优势也是劣势。按计划完成任务很好，但是真正的工作中有非常多的不确定性。

1 需求不确定：因为我们的分析阶段是自己调研的，而用户在真正接触到产品前，需求会发生变化，而且开发过程中会有很多的不确定因素，导致我们不能按照理想化的路走下去。

2 反馈太慢长：我们按照计划开发产品，A阶段完成后，只有到B阶段的时候才会检查A阶段工作结果的效果，效果反馈慢。而且有些错误是不可避免的，因为需求分析阶段是最容易出现错误的阶段，也就是说，我们会在错误的道路上走到最后(部署出来)才发现错了(需求错误)，这就意味着，当我们意识到错误的时候，已经迟了，我们需要花费很大的成本来修复错误。



3 缺乏并行性：每个阶段只接受上一阶段的成果后才工作，没有办法多阶段并行工作。

4 效率低下：A阶段在进行过程中，剩余的阶段都在等待，大量的资源(人力、物力)可能处于闲置状态，而且工作的人技术不一定很好。

场景：

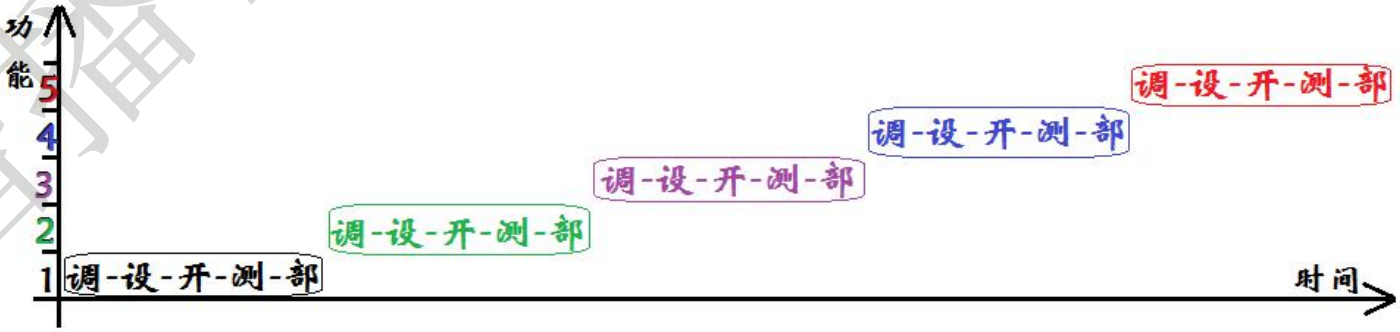
大部分的项目初期阶段

某些核心、大型、稳定性要求高的项目

4.1.3 迭代模型

迭代模型 (stagewise model)

瀑布模型出现的一段时间内，很多政府部门、企业都采用这种方式来开发自己的产品，由于产品规模越来越大，产品交付时间越来越短等因素，导致瀑布模型的劣势越来越突出，远远超出其创造的价值。这些劣势主要集中在不可控需求风险及产品交付周期上。



迭代模型是为了实现快速的将产品交付给用户，在设计产品的时候，不像瀑布模型那样设计的非常大/完美，而是一个阶段一个阶段的实现部分功能，最终交付给用户一个完善的产品。其中每一个阶段的功能都使用瀑布模型

开发，并且有一个可交付的产品成果。

优势：

1 反馈周期短：每个阶段的工作成果可以快速的交付给用户，用户接到产品后，使用的效果，都可以快速的反馈给产品人员。

2 降低产品风险：开发工作按照既定的计划进行推进，而且推进的过程中，可以结合上一个阶段的用户反馈来细化需求或者合理的变动部分功能/业务逻辑,并开始新一轮的迭代.

3 提高效率：阶段性的功能拆分及快速质量反馈，是的开发人员清楚产品的功能定位和问题焦点，工作效率会提高，加快整个项目工作的进度。

劣势：

团队水平：项目研发过程中，功能需求变动频繁导致风险增多，这对领导/组织者水平要求要高一些，软件研发团队的综合应变水平也有一定的要求。

场景：

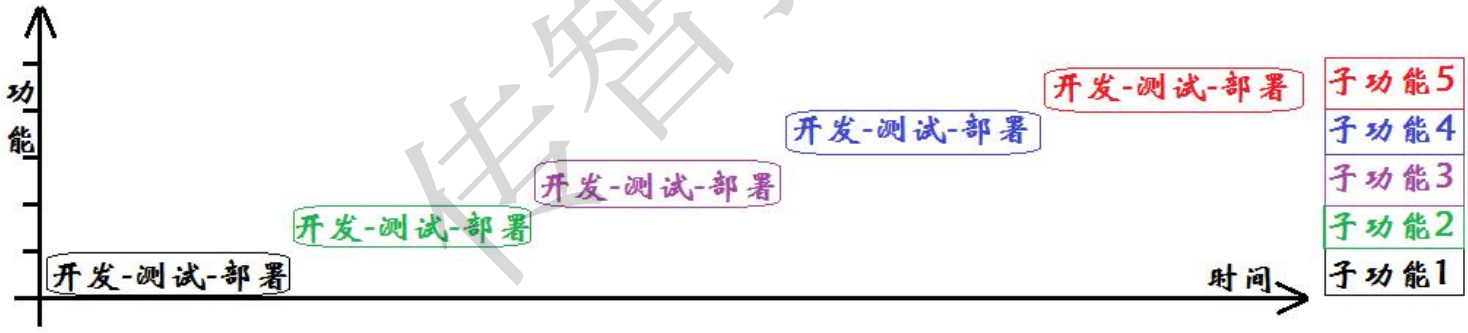
大部分项目的中后期的通用做法

高风险项目

4.1.4 增量模型

增量模型 (Incremental Model)

增量模型是一种分步开发的模型。它集成了瀑布模型的顺序特征和迭代模型的迭代特性。一般情况下，先针对一个大型的产品进行精细化设计，将复杂项目进行合理的阶段性功能拆分，然后每一个阶段的功能产品都使用瀑布模型开发，并且交付的子功能产品成果。每个阶段(B)都在前一个阶段(A)实现的功能基础上进行迭代开发，多个功能阶段迭代完毕后，就可以将最终完善的产品交付给用户了。



优势：

在保证项目目标的方向上，产品交付时间比瀑布模型短

在保证交付时间的标准上，产品功能目标比迭代模型好

劣势：

1 精细设计程度：在产品功能设计的时候，要把控好阶段性子功能的边界，对需求经常大变动的项目不太适合。

2 阶段性依赖：当前(B)阶段是前一个(A)阶段功能产品的基础上进行的，而且当前(B)阶段功能开发的过程中，不能破坏前一个(A)阶段的功能

3 团队水平：项目研发过程中，功能需求变动频繁导致风险增多，这对领导/组织者水平要求要高一些，软件研发团队的综合应变水平也有一定的要求。

场景：

大部分项目早期使用增量模型，可以规避技术风险。

交付时间紧张、人员不足的项目场景都可以。

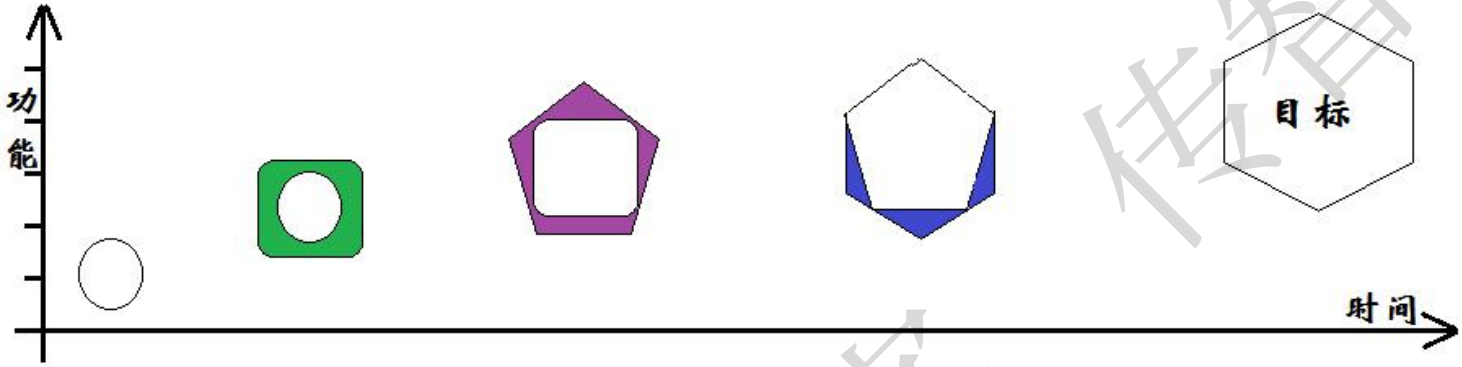
产品需求功能变动大的场景不太适合。

4.1.5 快速原型模型

快速原型模型 (Rapid Prototype Model)

快速原型这种模型一般使用于需求复杂/动态变化的软件系统。允许在调研阶段只对部分核心功能进行分析，用最少的时间做一个基本功能完备(可以运行)的产品原型，然后快速推上线，结合用户使用过程中具体的反馈信息，以丰富细化产品功能需求，在当前的产品原型基础上进行功能迭代更新,最终开发出客户满意的软件产品。

快速原型模型，又称原型模型，有点像“边做边改”与“迭代模型”的一个特殊的增量模型。以进化的开发方式为中心，迭代合理的新功能。



优势：

快速原型的目的在需求复杂不明的时候，通过快速的软件原型，理解和澄清问题，以便研发团队与用户达成共识，减少由于软件需求不明确带来的开发风险。

迭代新功能的时候会结合用户的反馈需求进行丰富细化，开发人员能确定客户的真正需求，所以该模型生产的产品一般比较能满足用户的需求。

劣势：

用户是善变的，开发会受到需求变更的影响，所以需要开发人员有快速重构项目的能力，应对各种未知风险的能力。

场景：

特殊的、需求不明的、复杂的项目，或者项目中的某些紧急bug。

4.1.6 螺旋模型

螺旋模型 (Spiral Model)

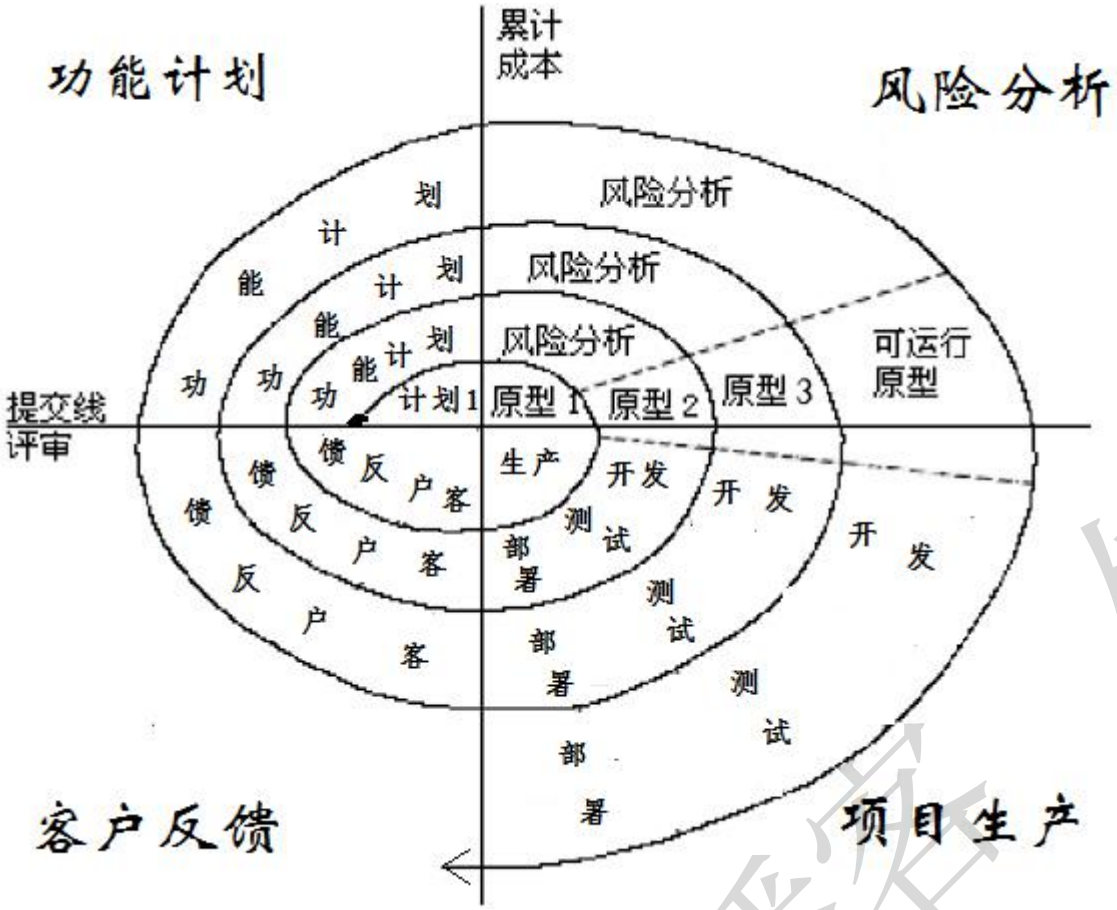
软件风险是任何软件开发项目中都存在的实际问题，而且项目越大，软件越复杂，风险就越大，这些风险在不同程度上损害项目产品的质量。所以，在项目研发过程中需要及时识别并分析风险，并采取适当措施以消除或减少风险的危害。

螺旋模型是在快速原型模型和瀑布模型的基础上，增加风险分析策略，结合多种方法尽量降低风险，保证项目的产品质量。当产品交付出去以后，在定制新需求前，评估之前的工作成果，然后进行新需求的风险分析，接着瀑布模型方式开发，直到交付出去产品，

螺旋模型沿着螺线进行若干次迭代，图中的四个象限代表了以下活动：

1 功能计划：确定软件功能，理清限制条件，制定研发方案
2 风险分析：评估研发方案，考虑如何识别和消除风险
3 项目生产：软件开发、测试和部署
4 客户反馈：工作复盘，提出修正建议，制定下一步计划。

传智播客



优势：

以进化的开发方式为中心，在每个项目阶段结合使用瀑布模型与快速原型模型。

强调了其他模型所忽视的风险分析

劣势：

强调风险分析，但要求许多客户接受并相信这种分析，是不容易的

场景：

特别适合于大型复杂的系统

4.1.7 敏捷开发模型

敏捷开发 (Agile Development)

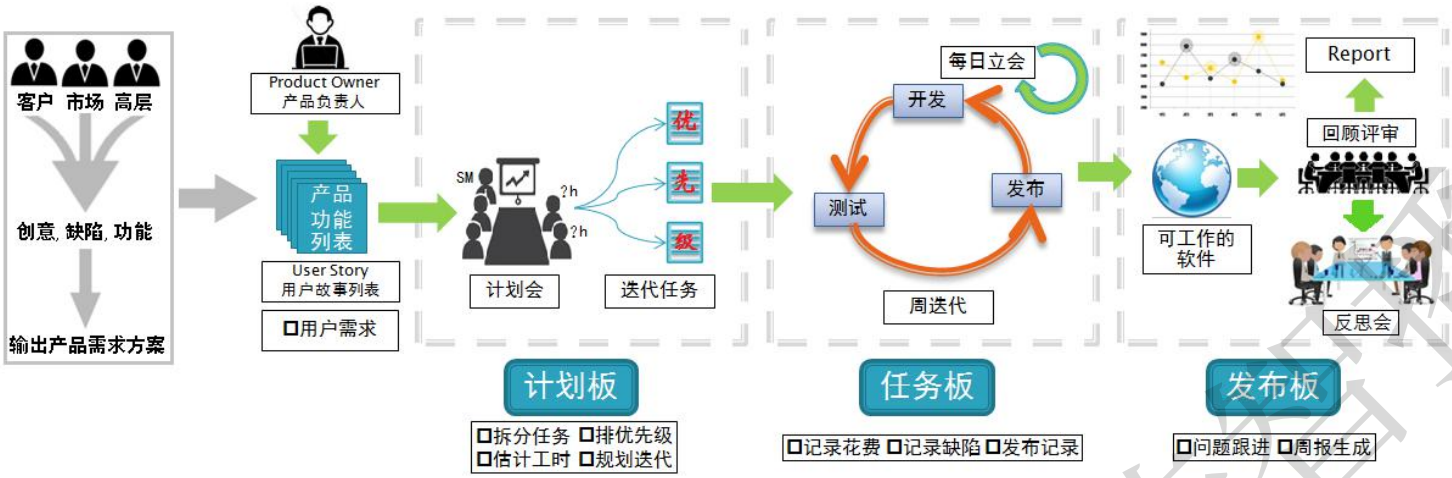
敏捷开发，是一种应对快速变化的用户需求的一种开发软件的**管理新模式**，其实它是XP、Scrum等数十种软件开发项目管理方法的集合，主要特点是：响应变更快、关注产品价值、注重个人的能力。我们从项目开发、功能迭代、团队沟通三个方面来学习。

项目开发：在敏捷开发中，最大的特点就是软件架构的解耦。也就是说：软件项目在初期被切分成多个相互联系，但也可独立运行的小项目，并分别完成，在整个软件开发过程中产品一直处于可使用状态。

功能迭代：强调较短的开发周期提交软件产品，相较于迭代模型更短(2-4周)。

核心特征：相较于个人(单团队)完成项目的传统软件开发模式(以文档方式推动项目前进)，敏捷开发更强调团队之间的紧密协作、团队小而精干，基于面对面的沟通，制定定迭代功能的优先级，能够很好地适应需求变化。

传智播客



优势：

产品团队、研发团队、测试团队之间更注重紧密协作，

团队小而精干，面对面(口头、源代码)交流，来深入理解产品的结构和功能。

开发内嵌测试，质量前置，等等

劣势：

团队的组建较难，人员不多，但综合技术能力要求较高。

场景：

项目复杂、交付周期短，功能迭代快的项目

4.1.8 精益模型

精益软件开发是精益制造原则和实践在软件开发领域的变体。它基于丰田生产方式(TPS)，由敏捷社区引入并发展。

精益原则：

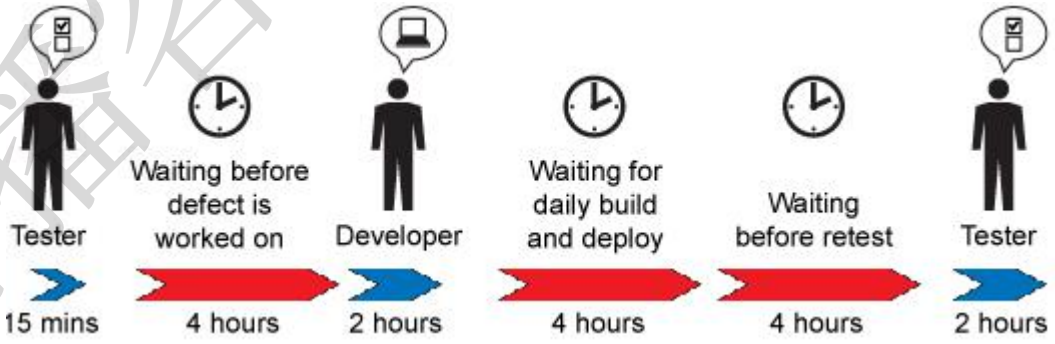
精益原则和精益制造原则的概念相近，精益开发也可以总结为如下七条原则：消除浪费、增强学习、下放权力、全局优化、延迟决策、内嵌质量、尽快发布

消除浪费

精益原则的中心思想就是为了消除浪费，首先必须能够识别、认识到浪费。按照精益思维，任何不能为客户增加价值的行为即是浪费。主要包括：不必要特性/需求/返工/延迟、需求不明、沟通低效等

以消除浪费和减少变异为目的。让资源能够投入到正确的地方提高质量，达到资源的高效率利用。

通过价值流的方法来消除浪费



团队措施：

增强学习

精益生产非常重视员工的培训，培训方式主要是通过做中学习进行的，而软件开发本身就是个持续学习的过程，通过多种实际的代码开发过程，可以培养并增强员工的技能。透过轮岗的方式使得员工取得多种技能。

在培养和提高员工技能的同时，也实现了人力资源的高效率利用。

下放权力

传统的团队里都是由团队的领导者来决定和分配每个人所要完成的任务。但是精益开发主张将这种权利下放到团队的每个人手里，通过频繁的沟通，让员工知道所有相关工作的全貌，而团队领导者提供团队成员应有的支持和帮助，克服困难，维持团队的合作默契。

全局优化

大多数的管理理论，强调工作拆解后，被分析出来各个子功能的最佳化。但从局部思考，常常会让整合的时候出现相依性的问题。精益生产鼓励人与人、团队之间的沟通，通过消除沟通过程中的隔阂和浪费，促进团队从不同面向去探讨整体产生最好的产品和服务呈献给客户。

项目措施：

延迟决策

软件开发的不确定因素很多，项目初期很难预测未来的变化，通过多种方法调研比较，尽可能的延迟决定，应该根据事实而非根据假设来做决策，才能应变不可避免的需求变更和修正错误。

系统越复杂，那么这个系统容纳变化的能力就应该越强，使其能够具备推迟重要以及关键的决定的能力。

内嵌质量

质量的管理在精益软件开发中尤其重要。质量应该贯穿在开发过程中的每一个阶段，当发生质量问题可以快速解决，从而提高客户良好的整体经验。

尽快发布

在一个技术发展非常迅速的时代，尽早的发布产品有助于更快的获得用户的反馈来改善当前产品的质量，从而更快的完成下一次功能迭代。越短的开发周期，能越快让开发团队从市场获得实时信息，应变市场的变化。

传智播客